MapAnything Vanilla Run

Before testing what wrong hints do to MapAnything, we checked that the plain, unmodified model runs correctly in our setup. This page shows what it built, the numbers, and what we fixed on the way.

SHORT ANSWER

Yes. Vanilla MapAnything ran properly.

- ✓ PASS It ran with no errors and produced sensible 3D on a real scene.
- ✓ PASS Correct hints made it clearly better, as the MapAnything paper says they should. Camera angles got 75% more accurate and the 3D shape 58% more accurate.
- ✓ PASS Running it twice gives exactly the same answer, down to the last digit.
- ✓ PASS The "answer key" (laser-scanned truth) agrees with itself on all 13 scenes (worst disagreement 0.21%).

What we asked the model to do

We gave MapAnything 10 photos of one place (a delivery area at ETH Zurich, from the ETH3D dataset) and asked it to rebuild the place in 3D and work out where each photo was taken from. ETH3D also has a laser scan of the same place, so we can measure how close the model gets.



The 10 input photos, each resized to 518 × 336 pixels, the size MapAnything works at.

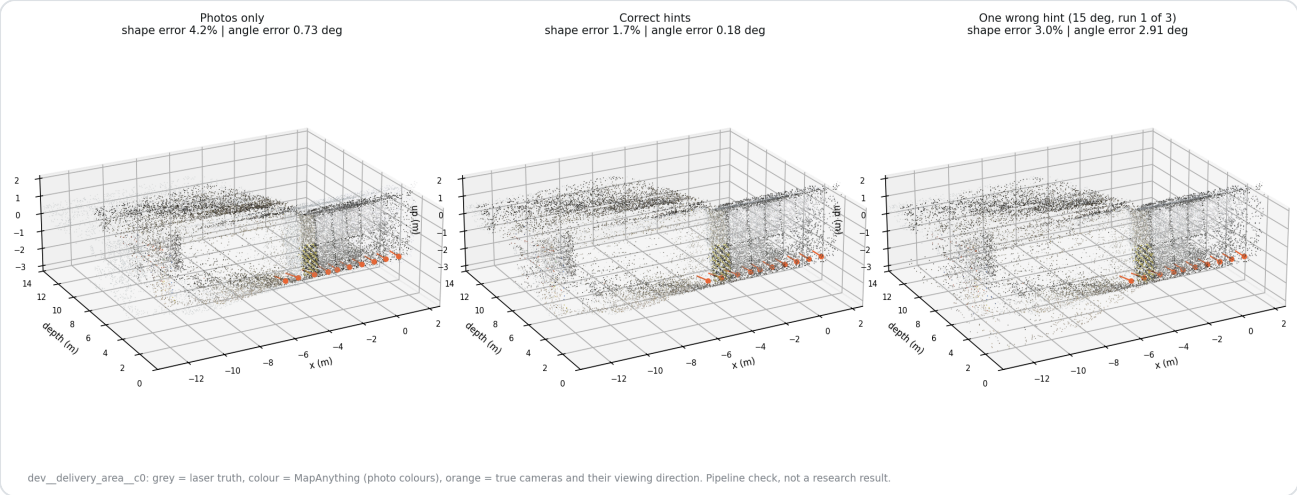
We ran it four ways:

- **Photos only.** The model has to guess everything.
- **Photos + lens info.** We also tell it each camera's focal length.
- **Correct hints.** Photos, lens info, and the true position of every camera.

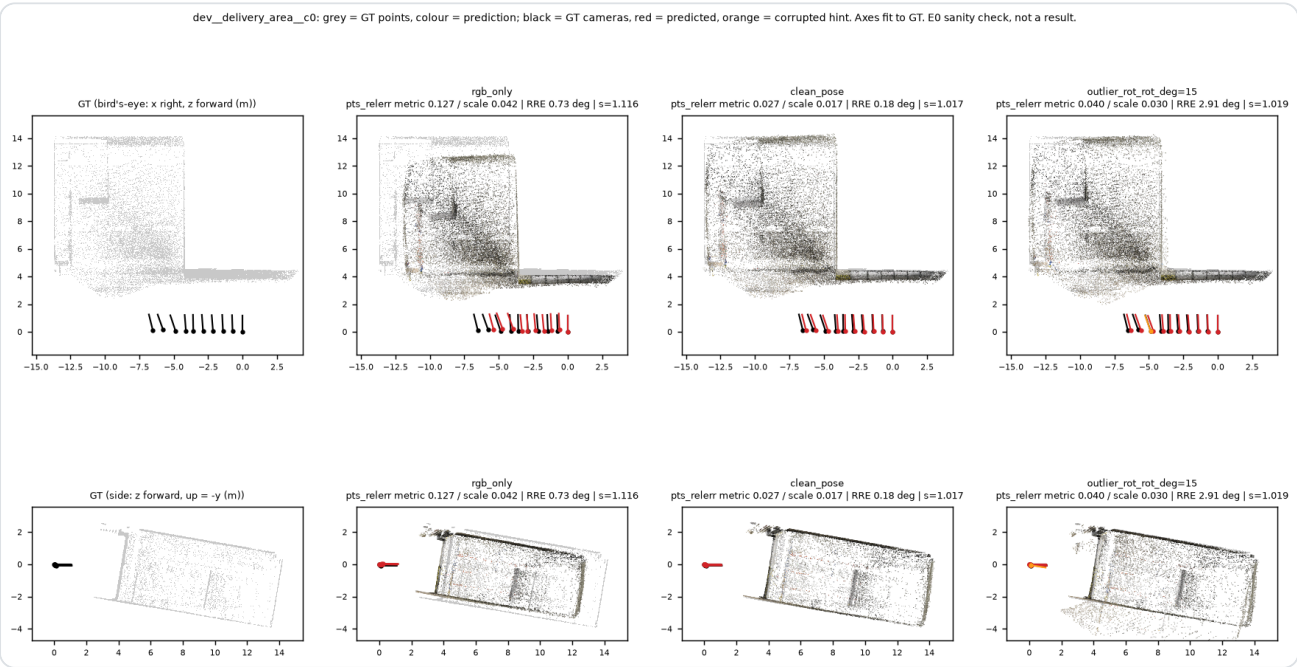
- **One wrong hint.** Same as correct hints, but one camera's direction is turned 15° wrong. This previews E1; here it only checks that the corruption code works.

The 3D reconstruction

In the HTML version this is an interactive 3D view you can turn and zoom; on paper it is shown as fixed views. Grey dots are the laser-scanned truth. Coloured dots are what MapAnything built, painted with the photo colours. Where the two overlap, the model got it right. Orange marks show the 10 true camera positions and the way each one faced.



Perspective views from just behind the camera row: photos only, correct hints, and one wrong hint (first of 3 runs).



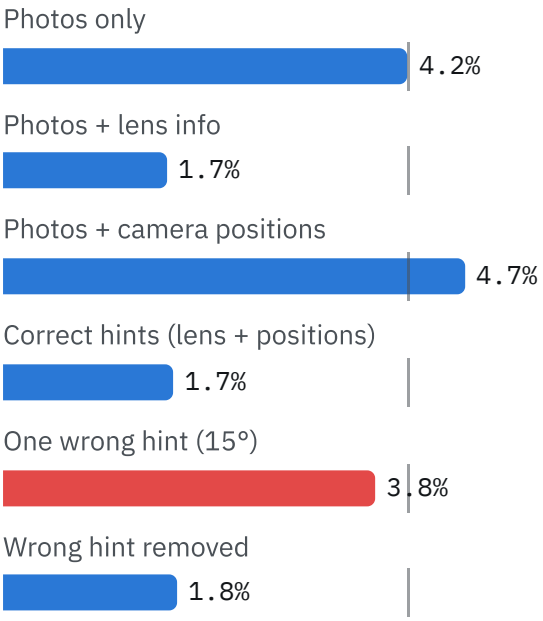
The same reconstructions flattened to a bird's-eye view (top) and a side view (bottom), with the model's own camera guesses. Black = true cameras, red = where the model thinks they are, orange = the wrong hint we gave it. With correct hints, red sits on black. With photos only, the model's sense of size is about 10% too small (scale factor s = 1.116), so its cameras sit too close together.

The numbers

Lower is better in every chart. The thin vertical line marks the photos-only result, so a bar that ends left of it beat "photos only". The wrong-hint rows are averages of 3 runs, each with a different camera turned wrong.

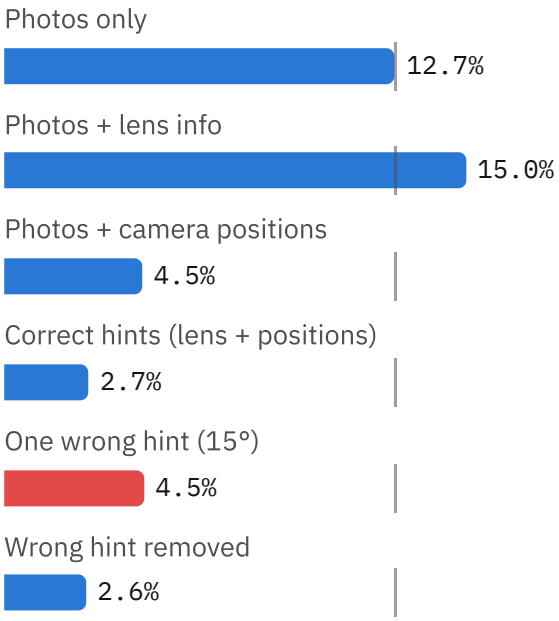
3D shape error

How far the 3D points are from the laser truth, after fixing overall size.



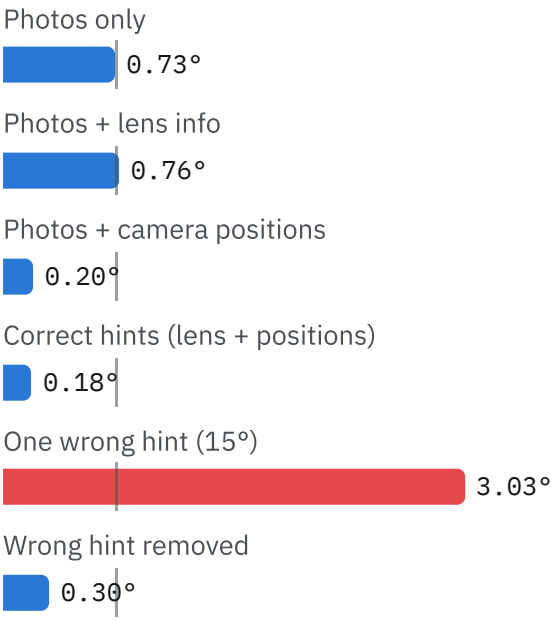
3D error at real size

The same, using the model's own guess of real-world metres.



Camera angle error

How wrong the model is about which way the cameras face, relative to each other.



All from one scene (`dev__delivery_area__c0`). This checks that the pipeline works. It is not yet a research result.
Sources: `results/smoke/per_run.csv`, `results/smoke_ctrl/per_run.csv`.

What the numbers say, in plain words

Correct hints help a lot

With the true camera positions, the 3D shape error fell from 4.2% to 1.7% and the camera-angle error from 0.73° to 0.18°. MapAnything is built to use hints like this, so this is the main sign that our pipeline feeds them in correctly. If we had a mistake in how camera positions are passed in, hints would have made things worse instead.

Lens info alone helps the shape, not the real-world size

Telling the model the focal length fixed the shape (4.2% to 1.7% error) but made its guess of real-world size slightly worse on this scene. That is one scene, so we only note it.

One wrong hint already hurts

Turning a single camera's hint by 15° raised the average camera-angle error to 3.0°, worse than photos only (0.73°). On this scene, one bad hint cost more than having no hints at all. This is exactly what E1 tests properly across many scenes. Here it only shows the test machinery works.

Removing the bad hint brings it back

When we dropped the wrong hint, errors went back to almost the correct-hint level (shape 1.8%, angle 0.30°). Our simple "compare with photos-only" filter found the bad camera by itself in all 3 runs. For a 15° error that is expected; harder cases come in E1.

Where the errors come from

"Vanilla" means we did not change the model. It does not mean the model is perfect. Every error number on this page measures the gap between what MapAnything built and what the laser scanner measured. That gap is what we want to measure. It is not a bug.

The model is an estimate

MapAnything is a neural network that guesses depth and camera positions from pixels. It is good, but not exact. Even with the true camera positions given, its 3D points are off by 1.7% on average.

Photos alone do not reveal size

From photos only, a wall 4 m away and 3 m tall looks the same as one 8 m away and 6 m tall. The model learns typical sizes from training and guesses. Here it guessed the scene about 10% too small. That is most of the 12.7% real-size error; once that single size factor is removed, 4.2% is left (the "shape" error).

Hints are inputs, not rules

MapAnything reads the camera positions as extra information but still predicts its own; it does not copy them. So even with correct hints its camera angles are 0.18° off.

> **Small images and hard surfaces**
>
> The model works at 518 × 336 pixels, so at 4-10 m one pixel covers roughly 1-3 cm of the scene. Large plain, shiny panels like the ones in these photos are hard to match from one photo to the next.

> **The answer key is almost exact**
>
> The laser data and camera calibration disagree with themselves by at most 0.21% (next section). That is far smaller than the model's errors, so it does not explain them.

> **Only the wrong-hint rows have error we added on purpose**
>
> Turning one camera by 15° is the test knob for E1. Every other row is the plain model on real data.

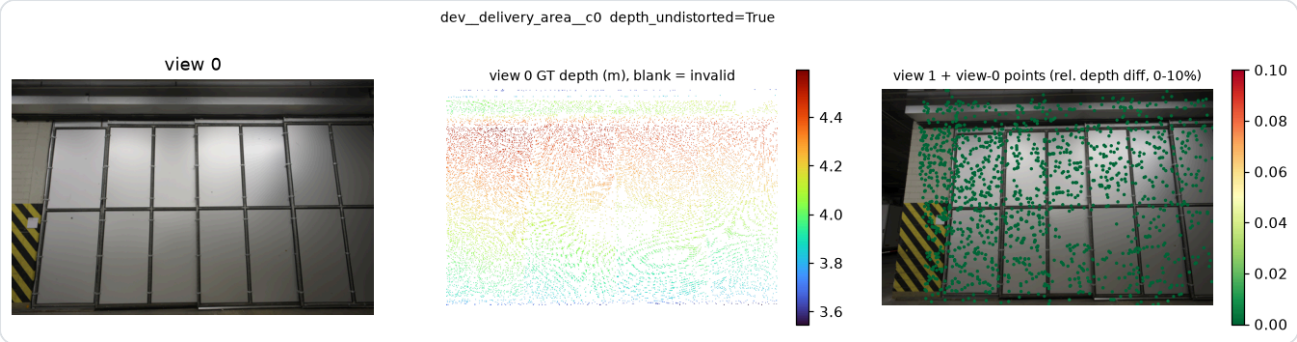
For scale: the MapAnything paper also reports a large drop when hints are added (relative point error from 0.08 to 0.02, on 2-photo tests averaged over three datasets). Our setup differs (10 photos, one scene, a different error formula), so the numbers are not directly comparable, but the direction matches. The two bugs we fixed in E0 were in our own data preparation; they made the run crash or mislabel scenes, and they do not affect these numbers.

Is the answer key right?

We grade the model against ETH3D's laser scans, so first we checked that the scans agree with themselves. We took the laser depth from photo 1, moved it into the view of the other photos using the true camera positions, and compared it with their own laser depth. If our camera handling were wrong, they would disagree by many percent.

Scene	Used for	Photo pairs checked	Worst disagreement	Result
delivery_area	practice (dev)	5 of 9	0.02%	✓ PASS
facade	practice (dev)	2 of 9	0.05%	✓ PASS
terrace	practice (dev)	3 of 9	0.21%	✓ PASS
courtyard	held-out test (eval)	3 of 9	0.05%	✓ PASS
electro	held-out test (eval)	3 of 9	0.20%	✓ PASS
kicker	held-out test (eval)	2 of 9	0.14%	✓ PASS
meadow	held-out test (eval)	5 of 9	0.18%	✓ PASS
office	held-out test (eval)	6 of 9	0.12%	✓ PASS
pipes	held-out test (eval)	5 of 9	0.12%	✓ PASS
playground	held-out test (eval)	3 of 9	0.09%	✓ PASS
relief_2	held-out test (eval)	9 of 9	0.10%	✓ PASS
relief	held-out test (eval)	9 of 9	0.09%	✓ PASS
terrains	held-out test (eval)	6 of 9	0.02%	✓ PASS

Pass rule: under 3% disagreement. Worst seen: 0.21%. Source: `reports/e0_clip_check/clip_check.csv`.



Left: photo 1. Middle: its laser depth (blank = no laser point; the scan covers about 1 pixel in 10). Right: those laser points moved into photo 2. Green means they land where photo 2's own scan says they should.

Speed, memory and repeatability

Time to rebuild 10 photos

0.26 s

on one A100 GPU, after a one-off 0.55 s warm-up

Peak GPU memory

8.3 GB

of the 80 GB card; a 24 GB card is plenty

Difference between repeat runs

0

bit-for-bit identical, so any change in E1 is real

Sources: `reports/e0_logs/7_smoke_check.txt`, `reports/e0_logs/8_det_b.txt`.

What we fixed on the way

> ### The laser depth was being matched to the wrong photos
>
> ETH3D's laser depth belongs to the original photos, which have lens distortion. The harness paired it with the straightened photos, which are even a different size (about 6200 × 4130 instead of 6048 × 4032). The original code crashed on the very first photo. We now re-map the depth through the lens model, the same way MapAnything's own ETH3D code does.

> ### Three test scenes were labelled as practice scenes
>
> When only some scenes were prepared, the code picked "practice" scenes from that subset instead of the fixed list. Three test scenes got the practice label. No model ever ran on them; we fixed the rule, added a test and rebuilt them.

What comes next

All 13 scenes are in and checked. The ETH3D server was down for about an hour during E0, so the last 3 scenes (relief, relief_2, terrains) were fetched later on the same machine.

Two decisions come before any E1 test-scene run. First, the relief and relief_2 clips are made entirely of photos that ETH3D stores turned sideways. Their geometry is correct, but MapAnything's own ETH3D code turns such photos upright, so we should decide whether to do the same. Second, decide whether

real-size errors are a main readout for the wrong-scale hints. Then come the new wrong-hint types for E1, with tests.

All numbers

Run	3D shape error	3D error, real size	Depth error, real size	Camera angle error	Camera direction error	Runs averaged
Photos only	4.2%	12.7%	10.4%	0.73°	3.15°	1
Photos + lens info	1.7%	15.0%	14.5%	0.76°	1.10°	1
Photos + camera positions	4.7%	4.5%	5.7%	0.20°	0.35°	1
Correct hints	1.7%	2.7%	2.0%	0.18°	0.31°	1
Correct hints, passed through the corruption code at 0°	1.7%	2.7%	2.0%	0.18°	0.31°	1
One wrong hint (15°)	3.8%	4.5%	2.4%	3.03°	1.46°	3
Wrong hint removed (we know which)	1.8%	2.6%	2.0%	0.30°	0.39°	3
Wrong hint removed (found automatically)	1.8%	2.6%	2.0%	0.30°	0.39°	3

"Shape" errors first remove one overall size factor per scene. "Real size" errors use the model's own metre-scale answer. Angle errors are averaged over all camera pairs. Internal names: rgb_only, rgb_K, clean_pose_noK, clean_pose, zero_noise, outlier_rot_rot_deg=15, drop_oracle, filter_rgbcheck.